

# AI 面试官语音转录项目

## 阶段性汇报、技术方案与部署说明

汇报日期：2026-08-19

项目仓库：RichardF123/AI-interviewer-speech-recognition

| 汇报重点 | 当前结论                                                          |
|------|---------------------------------------------------------------|
| 项目定位 | 电话式 AI 面试语音承载层，不绑定具体面试内容，可后续装载微调后的大模型。                        |
| 当前进度 | 已完成可本地运行 Demo：自动开麦、实时/准实时转写、DeepSeek 回复、语音播报、多轮输入优先处理。        |
| 核心风险 | 真正“豆包电话式”低延迟需要稳定流式 ASR/TTS。当前阿里云实时 ASR 在本机出现过 WebSocket 启动超时。 |
| 下一步  | 稳定实时 ASR，升级 LLM streaming 和流式 TTS，补充 VAD、打断和回声消除。             |

# 1. 项目目标

本项目的核心任务是开发一个可复用的 AI 语音转录与语音交互能力，面向“电话式 AI 面试”场景。用户点击开始后，应像打电话一样直接说话，系统实时显示候选人转写，大模型基于稳定文本生成回复，并自动读出语音。

- ☐ 候选人语音输入可以实时或准实时转成文字。
- ☐ 稳定 final 文本进入大模型，partial 只用于前端字幕展示。
- ☐ 大模型输出文字立即展示，并自动播报语音。
- ☐ 多轮对话不能被上一轮 LLM 或 TTS 卡住。
- ☐ 未来可直接替换为微调好的大模型，不被当前内容 prompt 绑定。

# 2. 当前阶段成果

| 模块           | 已完成能力                                               | 当前状态                 |
|--------------|-----------------------------------------------------|----------------------|
| 前端展示         | 电话式页面；开始面试后自动开麦；候选人输入区和 AI 输出区清晰分离。                 | 已完成                  |
| 实时字幕         | 浏览器 SpeechRecognition 可用时实时显示 partial/final。        | 可用，依赖 Chrome/Edge 支持 |
| 音频采集         | Web Audio API 采集 16kHz PCM，通过 WebSocket 上传。         | 已完成                  |
| LLM          | DeepSeek Chat Completions 接入；final 文本进入模型；多轮最新输入优先。 | 已完成                  |
| TTS          | 阿里云 TTS mp3 + 浏览器 speechSynthesis 兜底。               | 已完成                  |
| ASR Provider | 阿里云实时 ASR 主链路；MiMo 分段 ASR 备份。                       | 部分可用，需继续稳定实时 ASR     |
| 文档与版本        | README、前端/后端说明、技术报告已更新并推送 GitHub。                   | 已完成                  |

# 3. 技术方案

当前采用“前端实时字幕 + 后端语音网关 + 大模型 + TTS”的级联架构。这样做的好处是模块边界清晰，后续可以替换 ASR、LLM 或 TTS provider。

浏览器麦克风

- > Web Audio API / SpeechRecognition
- > WebSocket 事件: audio.chunk / speech.final
- > FastAPI Voice Gateway
- > DeepSeek 生成 assistant.text
- > 阿里云 TTS 或浏览器 TTS 播放

- ☐ partial 只展示，不进入 LLM，避免不稳定字幕污染模型上下文。
- ☐ final 才进入 LLM，并通过 llm.input 回传前端，便于排查模型到底收到什么。
- ☐ 多轮输入采用“最新 final 优先”，上一轮没完成时新输入可以取消旧任务。
- ☐ 系统状态、麦克风状态、网络状态不会作为候选人回答。
- ☐ TTS 不阻塞文字展示，外部 TTS 超时时浏览器 TTS 兜底。

## 4. 技术栈与解决的问题

| 技术栈                       | 用途            | 解决的问题                                    |
|---------------------------|---------------|------------------------------------------|
| React + Vite + TypeScript | 前端电话式交互页面     | 快速搭建可演示 UI，保证输入/输出状态清晰。                  |
| Web Audio API             | 麦克风采集与 PCM 处理 | 支持实时音频分片上传，为后端 ASR 做准备。                  |
| SpeechRecognition         | 浏览器实时字幕       | 在 Chrome/Edge 中提供低延迟 partial/final 文字显示。 |
| FastAPI + WebSocket       | 后端语音会话网关      | 承载 session、音频分片、final 文本、LLM/TTS 事件。     |
| DeepSeek Chat Completions | AI 面试官文字输出    | 接入真实大模型，替代 mock 回复。                      |
| 阿里云 NLS 实时 ASR            | 后端实时语音识别      | 目标是提供 provider 级 stt.partial/stt.final。  |
| MiMo ASR                  | 分段转写备份        | 在实时 ASR 不可用时支持音频片段转写，但不支持真正流式 partial。   |
| 阿里云 NLS TTS + Browser TTS | AI 回复语音播放     | 优先真实语音合成，失败时仍保证本地可听。                     |

## 5. 当前运行情况

- ☐ 本地前端运行地址： `http://127.0.0.1:5185/`
- ☐ 本地后端运行地址： `http://127.0.0.1:8011`
- ☐ 健康检查： `GET /health`
- ☐ Provider 状态： `GET /api/provider-status`
- ☐ 最新 GitHub main 提交包含代码和文档更新。

当前体验：Chrome/Edge 中打开页面，点击“开始面试”后，页面播放 AI 开场白，随后自动进入聆听。候选人说话时，浏览器实时字幕可马上显示；稳定 final 会进入 DeepSeek；AI 回复会展示并自动读出。

## 6. 已知问题与风险

| 风险         | 说明                                                 | 处理策略                                              |
|------------|----------------------------------------------------|---------------------------------------------------|
| 实时 ASR 不稳定 | 阿里云 NLS WebSocket 在本机测试中出现过 Wait response timeout。 | 确认项目权限、地域 endpoint、网络代理和防火墙；必要时更换流式 ASR provider。 |
| 内置浏览器限制    | Codex 内置浏览器可能不支持 SpeechRecognition。                | 正式演示使用 Chrome 或 Edge。                             |
| MiMo 非流式   | MiMo 当前文档为 wav/mp3 base64 分段识别。                    | 只作为备份，不作为电话式实时主链路。                                |
| LLM 非流式    | 当前 DeepSeek 调用为整段返回。                               | 下一步改成 streaming，首 token 立即显示。                     |
| TTS 非完全流式  | 阿里云 REST TTS 需要等文本片段完成。                            | 下一步接流式 TTS 或按句切分合成。                               |

## 7. 部署方案

部署分为前端静态站点和后端 API 服务两部分。前端可以部署到 GitHub Pages、Vercel 或任意静态站点平台；后端需要部署到能长期运行 WebSocket 的服务环境。

前端部署步骤：

- ☐ 进入 `app/frontend`，执行 `npm install` 和 `npm run build`。
- ☐ 产物在 `app/frontend/dist`。
- ☐ 生产环境配置 `VITE_API_BASE` 指向后端 HTTPS 地址。

后端部署步骤：

- ☐ 准备 Python 运行环境，安装 `app/backend/requirements.txt`。
- ☐ 配置环境变量：DeepSeek、阿里云 NLS、MiMo 等密钥。
- ☐ 用 Uvicorn/Gunicorn 或容器启动 FastAPI。
- ☐ 反向代理需支持 WebSocket，例如 Nginx。
- ☐ 生产环境必须使用 HTTPS/WSS，否则浏览器麦克风权限和 WebSocket 可能受限。

Frontend CDN / Static Hosting  
-> HTTPS 页面  
-> WSS 连接 Backend Voice Gateway  
-> ASR Provider / LLM Provider / TTS Provider  
-> 日志、指标、会话存储

## 8. 环境变量与 API 服务

| 服务       | 需要配置                                                                                    | 用途            |
|----------|-----------------------------------------------------------------------------------------|---------------|
| DeepSeek | DEEPSEEK_API_KEY, DEEPSEEK_BASE_URL, DEEPSEEK_MODEL                                     | AI 面试官文本生成。   |
| 阿里云 NLS  | ALIYUN_ACCESS_KEY_ID, ALIYUN_ACCESS_KEY_SECRET, ALIYUN-NLS_APP_KEY, ALIYUN-NLS_ENDPOINT | 实时 ASR 和 TTS。 |
| MiMo     | MIMO_API_KEY, MIMO_BASE_URL, MIMO_ASR_MODEL                                             | 分段 ASR 备份。    |
| 前端       | VITE_API_BASE                                                                           | 指定后端 API 地址。  |

密钥管理要求：真实 Key 只放部署平台环境变量或本地 .env，不能提交到 GitHub。

## 9. 下一阶段计划

- ☐ P0：修通并稳定阿里云实时 ASR，确保后端返回 stt.partial / stt.final。
- ☐ P0：增加前端更明确的 ASR 状态诊断，告诉用户当前是浏览器字幕、阿里云实时 ASR 还是分段备份。
- ☐ P1：DeepSeek 改为 streaming，首 token 立即显示。
- ☐ P1：TTS 改为句子级流式合成，减少等整段文本的延迟。
- ☐ P1：增加本地 VAD、回声消除和 barge-in，AI 播放中候选人说话可立即打断。
- ☐ P2：沉淀 provider 抽象，支持替换为微调模型或端到端实时语音模型。

## 阶段结论

当前项目已经从纯 mock 走到可运行的电话式语音 Demo：前端交互、DeepSeek、TTS 兜底、多轮输入稳定性都已打通。下一阶段的关键不是再改页面按钮，而是选择并稳定真正流式 ASR/TTS 能力，这决定它能否接近豆包电话式体验。